Department of Electrical and Computer Engineering
ECE 455 – Real Time Computer Systems Design Project
Section B03
Demo Date: March 25, 2026
Report Date: April 1, 2026

Project 2: Deadline-Driven Scheduler

Bashar Jirjees V00947950
Nicholas McLennan V00989854

Table of Contents

Table of Contents 1
1. Introduction and Design Document 1
1.1. Introduction 1
1.2. Design Document 2
2. Design Solution 3
2.1. Software Methodology 3
2.1.1. Sorting and Creating Tasks 3
2.1.2. Task Removal 5
2.1.3. Updating Task Priorities 6
2.1.4. Task Execution 7
2.1.5. Tasks Monitoring 8
2.1.6. Processing Messages 11
2.1.7. Main Task Scheduling 13
2.2. Software Diagrams 15
2.2.1. Tasks Communication 15
2.2.2. Tasks Monitoring 16
2.2.3. Deadline Driven Task Sorting Algorithm 17
2.3. Testing Methodology 18
2.3.1. Test Bench 1 (Under Utilization) 19
2.3.2. Test Bench 2 (Over Utilization) 20
2.3.3. Test Bench 3 (100% Utilization) 20
3. Discussion 21
4. Summary 22
5. Appendix A-1
6. References A-17

1. Introduction and Design Document

This section includes the general introduction of the deadline-driven scheduler (DDS) and an initial high level overview of the design with the design document.

1.1. Introduction

In this project, we implement the fundamental building blocks of a deadline-driven scheduler (DDS) where tasks (functions) are executed according to their absolute deadlines, ordered from the smallest to the largest. The project involves developing an API on top of the built-in functionality of FreeRTOS to ensure the orderly execution of tasks. Additionally, the system monitors task execution and completion times and identifies tasks that finish after their hard deadlines in order to evaluate the efficiency of the scheduler parameters.

Task preemption, priorities, and execution time slices are handled by FreeRTOS for simplicity. However, the selection of which tasks to execute and their classification based on deadlines and completion times are managed by an external algorithm developed from scratch. This algorithm operates through a message-based system in which different actions are performed depending on the message type, such as executing a released task, deleting a completed task, or monitoring overall system behavior.

The final version of the scheduler produced highly accurate results compared to the expected outcomes, but missed the deadlines in the provided test cases by 1–2 ms, primarily due to overhead introduced by real-time monitoring of the scheduler's behavior.

1.2. Design Document

Figure 1 shows the initial design document for the DDS. It highlights the FreeRTOS mechanisms including timers, queues, semaphores, and tasks. Timers are used to schedule the task releases and run the monitor task in intervals. Queues are used to interface between the DDS task and outside functions for task creation, monitoring, and completion. Task execution is handled by elevating the priority of FreeRTOS tasks with the earliest deadline. Tasks are stored in the DDS task using a linked list structure with the earliest deadline as the first element of the list. This list and task priorities are updated whenever a task is released or completed.

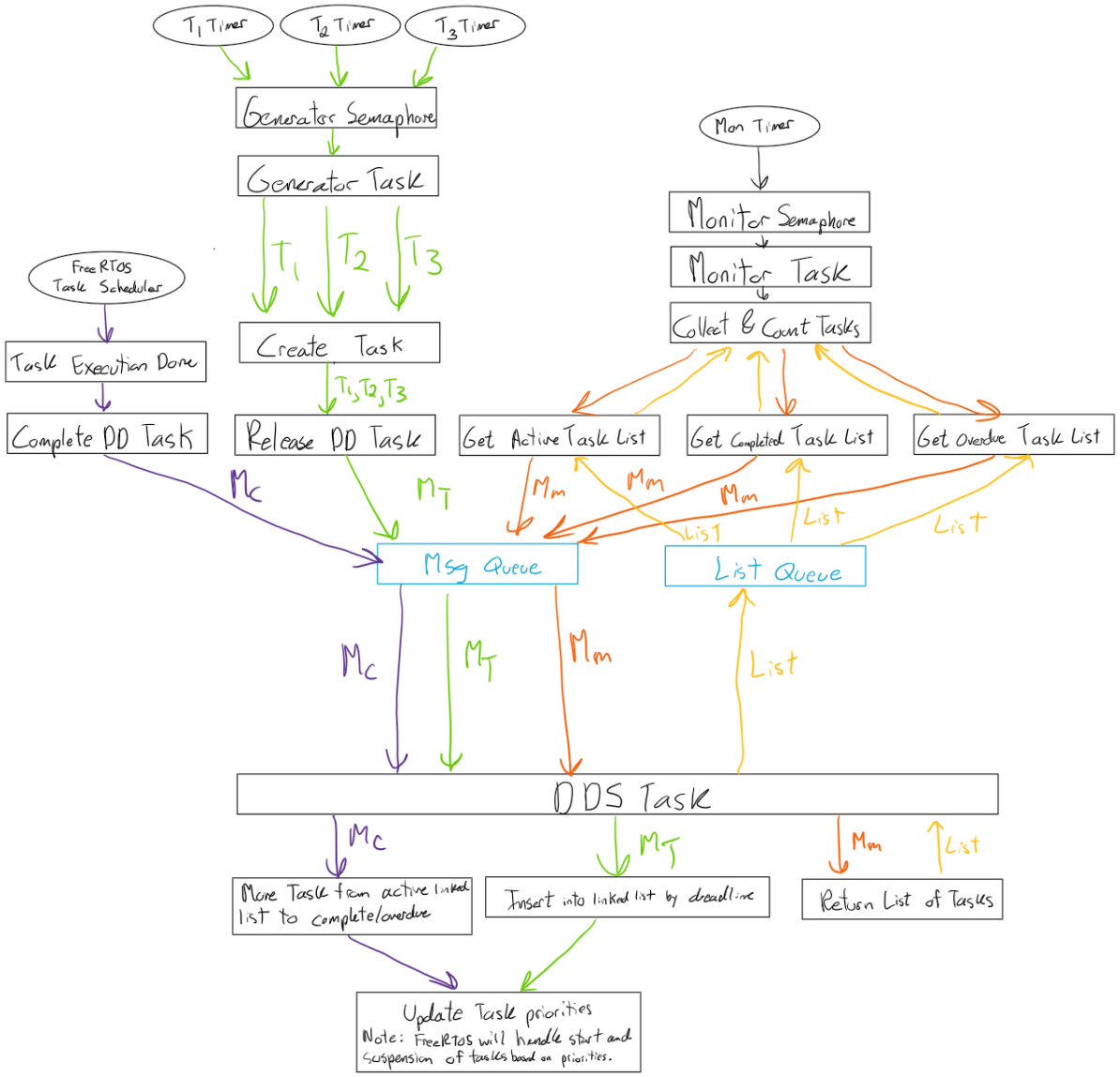

Figure 1: Initial Design Document for the DDS.

Many implementation details were required to fulfill the initial design document for the DDS, but the overall architecture remains the same. Details missing from the initial design document include how task execution is simulated, message structure, and memory allocation. These details are discussed in the Design Solution portion of the report.

2. Design Solution

This section includes the software design solution including code snippets and diagrams.

2.1. Software Methodology

This section includes the discussion of the implementation for the tasks, queues, and deadline driven scheduling algorithm.

2.1.1. Sorting and Creating Tasks

Newly created tasks are inserted into a linked list where their position is determined by their absolute deadline. Tasks with smaller deadlines are placed closer to the beginning of the list, while tasks with larger deadlines are placed further toward the end. Consequently, the task at index 0 has the earliest deadline and is executed first, whereas the task with the highest index has the latest deadline and is executed last. Task insertion follows a standard minimum-heap–like ordering process in which each new task is compared with existing tasks in the linked list until a task with a larger deadline is found, at which point the new task is inserted before it. As new task handles are created for each task, heap 4 must be selected in the FreeRTOS embedded environment.

Using a linked-list structure enables efficient task management because tasks can be inserted without shifting subsequent elements (tasks with later deadlines) or copying preceding elements (tasks with earlier deadlines), as would be required in an array-based structure. Instead, each task node stores pointers to other nodes in the list, allowing tasks to be added or removed with minimal memory overhead while maintaining the correct deadline ordering.

After newly generated tasks are inserted into the linked list, a message structure is used to assign a “Release” status to the task. This message is then placed into a global queue, from which tasks are retrieved sequentially. As tasks are removed from the linked list, the first task, corresponding to the smallest deadline is assigned the highest priority, while the remaining tasks are given a lower fixed priority. This ensures that the task with the earliest deadline executes first without interruption or preemption. To register the newly added task, a unique ID, release time, and deadline are needed for monitoring its execution.

```
void GenerateTasks(uint8_t task_id, uint32_t counter, uint16_t
                   chosen_test_executions[], uint16_t
                   chosen_test_periods[], task_type msg_type){
    task_params *task_params_temp = pvPortMalloc(sizeof(task_params));
    // For aperiodic, run in idle time or max deadline.
    uint16_t deadline = msg_type == PERIODIC ?
                        chosen_test_periods[task_id - 1] : 0xFFFF;
    xTaskCreate(Execute_Task, "task", 128, (void *) task_params_temp,
                0, &TaskHandle);
    vTaskSuspend(TaskHandle);
    create_dd_task(TaskHandle, msg_type,
              task_params_temp->task_counter_id,
              task_params_temp->taskid, task_params_temp->deadline);}
```

Code Sample 1: Task Creation Wrapper.

When a task is released, memory is allocated for its parameters and a new FreeRTOS task is created as shown in Code Sample 1. For aperiodic tasks, the deadline is unknown. Therefore, it is set to the maximum deadline as to be scheduled in idle time.

```
void create_dd_task(TaskHandle_t t_handle, task_type type, uint32_t
                    task_counter_id, uint32_t task_id, uint32_t
                    absolute_deadline) {
    task_message msg;
    msg.m_type = RELEASE;
    msg.t_handle = t_handle;
    msg.type = type;
    msg.task_id = task_id;
    msg.task_counter_id = task_counter_id,
    msg.absolute_deadline = absolute_deadline;
    xQueueSend(released_tasks_queue, &msg, pdMS_TO_TICKS(QUEUE_DELAY));
}
```

Code Sample 2: Creating Tasks

In Code Sample 2, task creation is handled by inserting a message structure into the released tasks queue. This message includes the task's release status ("RELEASE"), its type (periodic or aperiodic), and its original ID (a counter value used later for removal). For aperiodic tasks, the deadline is set to 0xFFFF or the maximum deadline supported by the DDS. This ensures it runs in idle time as the true deadline is unknown (Code Sample 3). It also contains a modified ID ranging from 1 to 3 to align with the test batch identifiers, a task handle for suspending the task or adjusting its priority, and a deadline, which for simplicity is assumed to be equal to the task's period in the test cases. Once the message structure is prepared, it is enqueued in the released tasks queue for further processing.

```
void insert_task(struct dd_task_list** head, struct dd_task_list*
                 new_task) {
    struct dd_task_list* current;
    if (*head == NULL || (*head)->task.absolute_deadline >
        new_task->task.absolute_deadline) {
        new_task->next_task = *head;
        *head = new_task;
    } else {
        current = *head;
        while (current->next_task != NULL &&
               current->next_task->task.absolute_deadline <=
               new_task->task.absolute_deadline) {
            current = current->next_task;
        }
        new_task->next_task = current->next_task;
        current->next_task = new_task;
    }
}
```

Code Sample 3: Task Insertion and Sorting

In Code Sample 3, whenever a new task is created, it is inserted into the active tasks list (a linked list) according to its absolute deadline. The task is positioned such that the preceding task has a lower or equal deadline, while the following task has a higher deadline. If no such position is found, the task is appended to the end of the list. Additionally, if the new task's absolute deadline is smaller than that of the current first task, it is inserted at the beginning of the list.

2.1.2. Task Removal

After a released task completes execution, it is removed from the active task list and returned for completion-time verification. The removal process uses the task's ID to locate it in the list; once found, it is bypassed by linking its predecessor directly to the following task. The removed task is then fully detached from the active list.

If the task to be removed is the first element, the second element becomes the new head of the list, and the original first element is returned.

```
struct dd_task_list* remove_task(struct dd_task_list** head, uint32_t
                                 id) {
    struct dd_task_list* temp = *head;
    struct dd_task_list* prev = NULL;
    if (temp != NULL && temp->task.task_id == id) {
        *head = temp->next_task;
        temp->next_task = NULL;
        return temp; }

    while (temp != NULL && temp->task.task_id != id) {
        prev = temp;
        temp = temp->next_task; }
    if (temp == NULL) return NULL;
    prev->next_task = temp->next_task;
    temp->next_task = NULL;

    return temp; }
```

Code Sample 4: Task Removal

In Code 4 Sample, the task id is passed and the task counter id is checked against every element in the active list and is removed by linking the previous task to the one following the removed. The removed task is removed and is unlinked from the active task list by setting the next element linked to NULL.

```
void delete_dd_task(uint32_t task_counter_id, uint8_t task_id) {

    uint32_t current_time = xTaskGetTickCount();

    task_message msg;

    msg.m_type = COMPLETE;
    msg.task_id = task_id;
    msg.completion_time = current_time;
    msg.task_counter_id = task_counter_id;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(QUEUE_DELAY));
}
```

Code Sample 5: Flagging Task as Completed

Before removing a task from the active task list upon completion, its completion time is recorded. A message structure with a "COMPLETE" flag is then added to the task queue; this message is later checked when dequeued, allowing the completed task's information to be displayed.

It is important to note that each task has two identifiers: one ranging from 1 to 3, representing the task ID within the test batch, and another unique counter ID assigned to each task instance. The counter ID is particularly important, as it is used to accurately identify and remove the correct task from the active task list.

2.1.3. Updating Task Priorities

After tasks are added to the task queue based on their timers, the newly created tasks are inserted with a “RELEASE” status. They are then moved into the active task list, where they are sorted in ascending order of their deadlines.

Once all tasks have been added to the queue for a given time slice, the task with the earliest deadline (i.e., the first task in the active task list) is assigned the highest priority and is resumed or started. All other tasks, which have equal or later deadlines, are assigned the lowest priority and are suspended. This ensures that only one task executes at a time and prevents it from being preempted by other tasks in the active task list.

```
void update_tasks_priorities(void) {

    struct dd_task_list* current = active_list_head;
    if (current != NULL) {

        vTaskPrioritySet(current->task.t_handle, HIGH_PRIORITY);
        if(eTaskGetState(current->task.t_handle) == eSuspended ||
           eTaskGetState(current->task.t_handle) == eBlocked)
                 vTaskResume(current->task.t_handle);

        current = current->next_task;
        while (current != NULL) {

            vTaskPrioritySet(current->task.t_handle, MIN_PRIORITY);
            vTaskSuspend(current->task.t_handle);

             current = current->next_task;
        }
    }
}
```

Code Sample 6: Updating Tasks’ Priorities

In Code Sample 6, the task at the head of the active task list is assigned the highest priority, while all other tasks in the linked list are assigned a priority of 0, the lowest possible value, and are suspended. This ensures that they do not preempt the highest-priority task. If the highest-priority task is suspended—such as when a new task with an earlier deadline is added and becomes the next highest-priority task—the currently running task is also suspended and its priority is reduced to 0. It will resume execution later when priorities are recalculated as new tasks continue to be generated.

2.1.4. Task Execution

Tasks execution time is set within the test batches provided, and they are simulated through a unique delay for each task. The executing task within that timeslice is always the task with

highest priority/lowest deadline within the active tasks list and the task and can be interrupted only if a new task is generated and and has higher priority based on its deadline within the delay of the already executing task.

```
void Execute_Task(void *pvParameters) {

    task_params *task_params_temp = (task_params *)pvParameters;

    uint32_t task_counter_id = task_params_temp->task_counter_id;
    uint32_t start_time = xTaskGetTickCount();
    uint32_t release_time = task_params_temp->release_time;
    uint32_t task_deadline = release_time + task_params_temp->deadline;
    uint16_t ticks_counter = 0;
    uint8_t task_id = task_params_temp->taskid;

    taskENTER_CRITICAL();

    printf("\nT:%i, Stat:%s, RT:%i, ST:%i, DT:%i, CT:%i\n",
                                    (int)task_id,
                                    "Active",
                                    (int)release_time,
                                    (int)0,
                                    (int)task_deadline,
                                    0);
    taskEXIT_CRITICAL();

    uint32_t endtime = xTaskGetTickCount();
    uint32_t prev_tick = xTaskGetTickCount();

    // Ensuring that the Number of Ticks Between Start Time and Finish
    // Time is Always Bigger or Equal to the Task Delay
    while(1) {

        uint32_t tick = xTaskGetTickCount();

        if (tick > prev_tick) ++ticks_counter;

        prev_tick = tick;



        // Ensuring the Total Delay is Accounting for the Delay of
        // Printing the Task Stats in the Prior "Printf" Statement

      if (ticks_counter >=  (task_params_temp->delay - (endtime -
          start_time))) break;
     };

    // Getting the Real Task Completion Time
    uint32_t delay = xTaskGetTickCount();

    //Flag The Task For Deletion After Completion
    delete_dd_task(task_counter_id, task_id, delay, task_deadline);

    taskENTER_CRITICAL();

    printf("\nT:%i, Stat:%s, RT:%i, ST:%i, DT:%i, CT:%i\n",
```

```
                                (int)task_id,
                                completed_overdue[task_deadline >=
                                                  delay ? 0 : 1],
                                (int)release_time,
                                (int)start_time,
                                (int)task_deadline,
                                (int)delay);
    taskEXIT_CRITICAL();

    // Delete the Task After Completion
    vTaskDelete(NULL);

    vPortFree(task_params_temp);
}
```

Code Sample 7: Task Execution

Code Sample 7 demonstrates that once the highest-priority task begins execution, the current system time is recorded as its release time. In addition, the task's ID and unique counter ID are stored, which are required for displaying the task information and correctly removing it upon completion. The task ID (ranging from 1 to 3), active status, start time, and expected deadline are also displayed to ensure accurate tracking of execution, as well as proper scaling of release times and deadlines based on the current system time and previously executed tasks.

After the task completes its delay/execution, it is marked as complete and added back to the task queue for further processing. The completed task is then removed from the FreeRTOS registry to ensure it is not scheduled again unless a new instance of the same task is generated in a future period. The task execution time is tracked using a tick counter that begins at zero and increments until it reaches (or exceeds) the specified delay. This approach ensures the delay remains consistent, regardless of whether the task is preempted, since the tick count progresses in sync with the system time.

When a task begins execution, its parameters, such as active status, release time, start time, and deadline, are displayed for monitoring purposes. The start time remains at zero during execution and is only updated once the task completes. At that point, the task status is set to either complete or overdue, and the final values of start time, release time, completion time, and deadline are displayed.

2.1.5. Tasks Monitoring

A task monitor is used to track the number of the active, completed, and overdue tasks every 200 ms to determine the accuracy of tasks execution periods within 1.5 second intervals/hyperperiods, ensuring the tasks behaviour is matching the expected performance.

```
struct dd_task_list* get_active_dd_task_list(void) {

    task_message msg;
```

```
    struct dd_task_list* active_list_head;
    msg.m_type = ACTIVE_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &active_list_head,
                  pdMS_TO_TICKS(100));

    return active_list_head;
}

/*
    This Function Returns the List of the Currently Completed Tasks

    Input: None
    Return: The Completed Tasks List Head
*/
struct dd_task_list* get_completed_dd_task_list(void) {

    task_message msg;
    struct dd_task_list* completed_list_head;
    msg.m_type = COMPLETED_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &completed_list_head,
                  pdMS_TO_TICKS(100));

    return completed_list_head;
}

/*
    This Function Returns the List of the Currently Overdue Tasks

    Input: None
    Return: The Overdue Tasks List Head
*/
struct dd_task_list* get_overdue_dd_task_list(void) {

    task_message msg;
    struct dd_task_list* overdue_list_head;
    msg.m_type = OVERDUE_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &overdue_list_head, pdMS_TO_TICKS(100));
    return overdue_list_head;
}
```

Code Sample 8: Getting Total Tasks Counts within a List

In Code Sample 8, **get_active_dd_task_list()** function returns the list of the currently active tasks which haven't completed execution or the tasks that have just been released. The **get_completed_dd_task_list()** returns the list of the tasks that have completed execution since the scheduling process started from time 0. The **get_overdue_dd_task_list()** returns the list of the tasks that have finished execution after their specified deadline since the scheduler started at time 0.

All the functions return the list of the task within each state of active, completed, overdue by adding the list head (linked-list) of the target tasks to a separate lists queue that would release the lists head structure when they are requested by the monitor task.

```
void get_current_tasks(struct dd_task_list* head, char* task_status) {

        struct dd_task_list* current = head;

        uint32_t count = 0;

        char *total_count_status[] = {"GET_ACTIVE_TASK_COUNT",
             "GET_COMPLETED_TASK_COUNT", "GET_OVERDUE_TASK_COUNT"};

        while (current != NULL)
        {
            // Checking that the Semaphore is Available Before
            // Increment Over Counting
            ++count;
            current = current->next_task;
        }

       if (strcmp(task_status , total_count_status[0]) == 0)
           printf("A#: %i\n", (int)count);
       else if (strcmp(task_status , total_count_status[1]) == 0)
           printf("C#: %i\n", (int)count);
       else if (strcmp(task_status , total_count_status[2]) == 0)
           printf("O#: %i\n", (int)count);
}
```

Code Sample 9: Tasks Counter

In Code Sample 9, the number of tasks within the active/completed/overdue tasks list is counted upon each call to the get_current_tasks() with the tasks list head (linked-list) and the list tasks type to select the different output associated with each count.

```
void vMonitorTask(void *pvParameters) {

    Mutex_Monitor_Timer = xSemaphoreCreateBinary();

    // Timer is Used to Report the Total Number of Tasks Every 200
    // Milliseconds
    TimerHandle_t xTimer_Total_Counts =
    xTimerCreate("Total_Counts_Timer", pdMS_TO_TICKS(200),
                                                    pdTRUE, NULL,
    vTotal_Counts_Timer_Callback);

    xTimerStart(xTimer_Total_Counts, 0);
    while (1) {

        //Checking the Timer has Been Triggered
        if (xSemaphoreTake(Mutex_Monitor_Timer,
            pdMS_TO_TICKS(SEMAPHORE_DELAY)) == pdTRUE){

            printf("\n");
            get_current_tasks(get_active_dd_task_list(),
            "GET_ACTIVE_TASK_COUNT");
            get_current_tasks(get_completed_dd_task_list(),
```

```
            "GET_COMPLETED_TASK_COUNT");
            get_current_tasks(get_overdue_dd_task_list(),
            "GET_OVERDUE_TASK_COUNT" );
        }
    }
}
```

Code Sample 10: Monitor Task

In Code Sample 10, the monitor task uses a 200 ms timer that periodically unblocks the printing of the total number of active, completed, and overdue tasks in their respective lists. When the timer expires, it releases a semaphore, which unblocks the infinite loop and triggers the display of these counts. The counts are obtained by calling the get_current_tasks() function with the corresponding list head (linked-list) and a type identifier "GET_ACTIVE_TASK_COUNT", "GET_COMPLETED_TASK_COUNT", or "GET_OVERDUE_TASK_COUNT" to indicate which task category should be reported.

2.1.6. Processing Messages

Newly generated tasks from the task generator are packaged into a message structure labeled “RELEASE.” This structure includes the task ID (ranging from 1 to 3), a unique task counter ID, task handle, release time, periodicity type (periodic or aperiodic), and deadline. These parameters are also stored within a dedicated task handle, after which the task is inserted into the active task list in ascending order of deadlines. Once added, the priorities of all active tasks are updated so that the task with the earliest deadline is selected for execution.

When a task completes execution, a corresponding “COMPLETE” message is created. The task is then removed from the active list and placed into either the completed or overdue list, depending on whether it finished before or after its deadline. Task priorities are subsequently recalculated to ensure that the next highest-priority task is scheduled for execution.

Once messages with “ACTIVE_TASKS”, “COMPLETED_TASKS”, “OVERDUE_TASKS”, is received , the head of the active, completed and overdue tasks list is sent to a separate lists queue where the counts will be processed when monitoring the tasks.

```
void vDDSTask(void *pvParameters) {

    task_message msg;
    uint32_t current_time;

    while (1) {


        if (xQueueReceive(tasks_queue, &msg,
            pdMS_TO_TICKS(QUEUE_DELAY)) == pdTRUE){

                current_time = xTaskGetTickCount();


                if(msg.m_type == RELEASE) {

                    struct dd_task_list* new_task =
                            pvPortMalloc(sizeof(struct dd_task_list));
```

```
                    new_task->task.t_handle = msg.t_handle;
                    new_task->task.type = msg.type;
                    new_task->task.task_id = msg.task_counter_id;
                    new_task->task.release_time = current_time;
                    new_task->task.absolute_deadline = current_time +
                    pdMS_TO_TICKS(msg.absolute_deadline);
                    new_task->task.completion_time = 0;
                    new_task->next_task = NULL;

                    insert_task(&active_list_head, new_task);

                    if(xQueuePeek(tasks_queue, &msg, pdMS_TO_TICKS(0))
                       == pdFALSE)
                       update_tasks_priorities();
                }

                else if(msg.m_type == COMPLETE) {

                    struct dd_task_list* completed_task =
                    remove_task(&active_list_head,
                                msg.task_counter_id);

                    if (completed_task != NULL){

                        completed_task->task.completion_time =
                        msg.completion_time;

                        if (completed_task->task.completion_time >
                            msg.absolute_deadline) {
                            completed_task->next_task =
                                            overdue_list_head;
                            overdue_list_head = completed_task;

                        }else {
                            completed_task->next_task =
                                            completed_list_head;
                            completed_list_head = completed_task;
                        }
                    }
                    update_tasks_priorities();

            }else if(msg.m_type == ACTIVE_TASKS) {

                xQueueSend(lists_queue, &active_list_head,
                pdMS_TO_TICKS(0));
            }else if (msg.m_type == COMPLETED_TASKS) {
                xQueueSend(lists_queue, &completed_list_head,
                pdMS_TO_TICKS(0));
            }else if (msg.m_type == OVERDUE_TASKS) {
                xQueueSend(lists_queue, &overdue_list_head,
                pdMS_TO_TICKS(0));
            }
        }
    }
}
```

Code Sample 11: Message Processing

In Code Sample 11, messages are processed after being dequeued from the main task queue and examined based on their type. This continues until the queue becomes empty or the time slice allocated to the `vDDTask()` expires under the built-in Round-Robin scheduler in FreeRTOS. A temporary message structure is used to hold the current message, which is then either used to initialize a new task structure or to store parameters of completed tasks for status evaluation.

If the message type is “RELEASE,” the current time is recorded as the release time, indicating that the task is ready for execution, although it may not begin immediately if a higher-priority task is currently utilizing the required resources.

2.1.7. Main Task Scheduling

The message processing, task generator, and monitoring tasks are all assigned the highest priority to prevent disruptions under varying system loads. Without this, heavy load conditions or frequent preemption could cause these functions to be skipped, leading to issues such as missed task generation intervals, interrupted periodic monitoring of task counts, and failure to process newly created tasks in a timely manner. All tasks and their priority are summarized in Table 1.

FreeRTOS Task	Priority
vDDTaskGenerator	MAX_PRIORITY
vDDSTask	MAX_PRIORITY
vMonitorTask	MAX_PRIORITY
Generated Task (Earliest Deadline)	HIGH_PRIORITY
Generated Task (Not Earliest Deadline)	MIN_PRIORITY

Table 1: List of all F-Tasks and their priority levels.

```
int main(void) {

    tasks_queue = xQueueCreate(10, sizeof(task_message));
    lists_queue = xQueueCreate(10, sizeof(struct dd_task_list));

    if (tasks_queue != NULL && lists_queue != NULL)
    {

        printf("\nTasks Monitor:\n");

        //Defining and Initiating the Tasks for FreeRTOS
        xTaskCreate(vDDTaskGenerator, "Task_Generator", 256, NULL,
        MAX_PRIORITY, &generator_handle);
        xTaskCreate(vDDSTask, "Task_Checker", 128, NULL,
        MAX_PRIORITY, &dd_task_handle);
        xTaskCreate(vMonitorTask, "Task_Monitor", 128, NULL,
        MAX_PRIORITY, &monitor_handle);
```

```
        vTaskStartScheduler();
    }

    while(1);

    return 0;
}
```

Code Sample 12: Main Tasks Scheduling

In Code Sample 12, the main task queue is defined to store messages that manage the processing of newly generated and completed tasks, as well as control the execution of the monitoring process. Additionally, a separate queue for task lists is established to store the heads of the active, completed, and overdue linked lists, which are retrieved when the monitor task begins execution. Both the main task queue and the lists queue are configured with a size of 10 to reduce memory usage.

It is worth noting that both the main tasks queue takes a message structure that stores the message type and task parameters. The lists queue takes a dd_task_list which represents the data type of the head element of the active/completed/overdue linked lists.

The memory allocated to the task generator is twice that of the monitor and task-processing components because it stores the test batches, copies the selected batch into a separate array, creates three unique timers for each task in the batch, and keeps track of the unique ID for each periodically released task.

2.2. Software Diagrams

This section includes the diagrams for overall task communication, the monitor task, and earliest deadline first sorting algorithm.

2.2.1. Tasks Communication

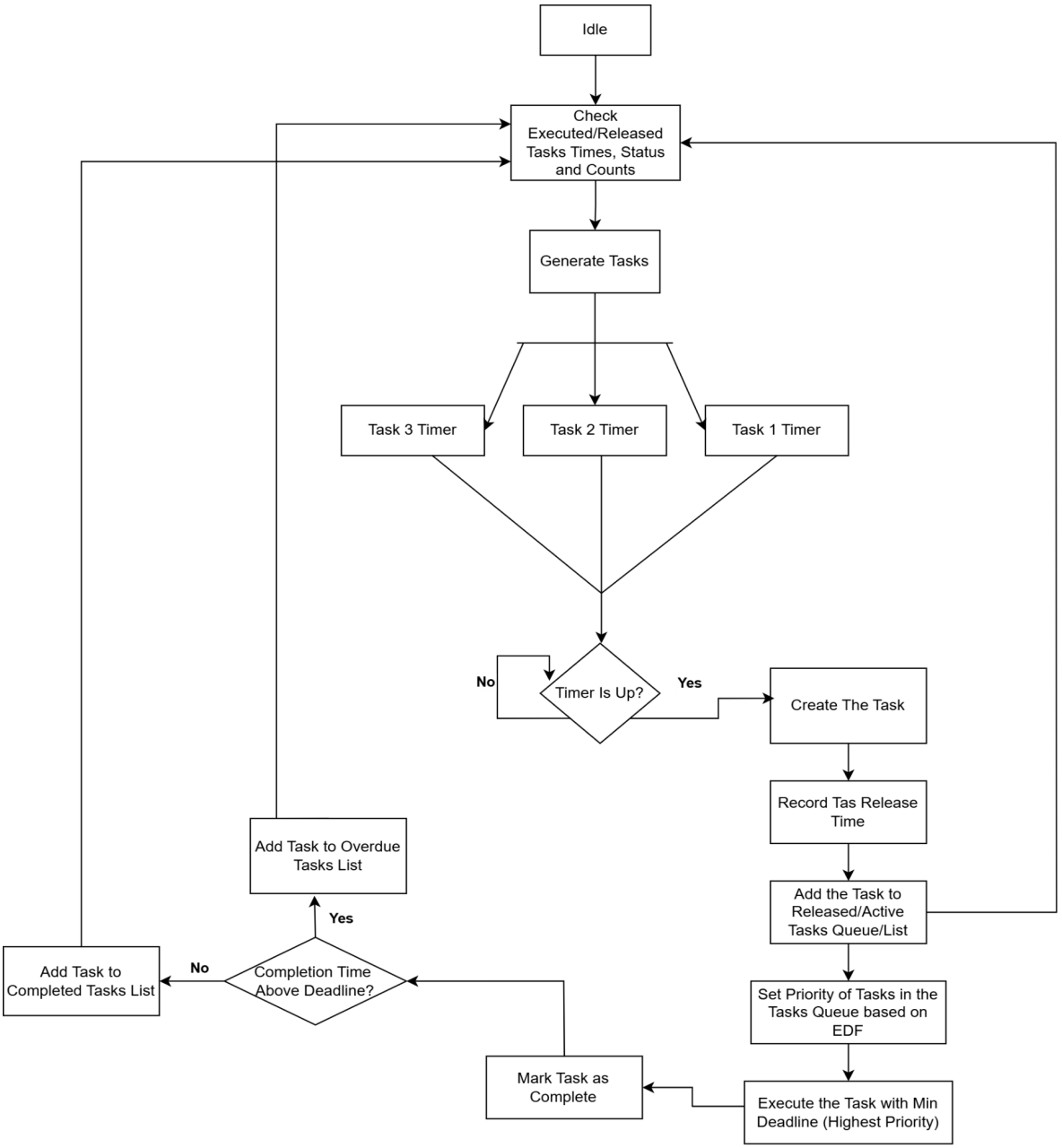

Diagram 1: Tasks Communication Flow Chart

The system begins in the idle state. It first performs a status check on the current active, completed, and overdue tasks, since this is the highest-priority function. It then attempts to generate new tasks at specified time intervals, where each task period, meaning the frequency at which the task reoccurs, is equal to its deadline. These task periods are defined according to the test batches provided for the three tasks. After the correct timer wait time is

set for each period, the timers begin counting down, and when a timer expires, the corresponding task is released and added to the released tasks queue.

Once tasks are added to the released tasks queue, they are sorted by the earliest deadline and inserted into the active tasks list in ascending order. The task with the smallest deadline or period is executed first by giving it the highest priority, while the remaining tasks in the list are suspended.

After a task finishes executing, its completion time is recorded and compared with its deadline to determine whether it is overdue, meaning it was completed after the deadline specified in the test batch, or not. The completed task is then added to the completed or overdue tasks list based on its status and removed from the active tasks list. The completed/overdue tasks list supports insertion only, so tasks cannot be removed from it; this allows the scheduler to keep track of how many tasks have completed within their deadline since it started. After each task completes execution, the current task status is printed to the console, and the timers are checked again in case a new task must be released.

2.2.2. Tasks Monitoring

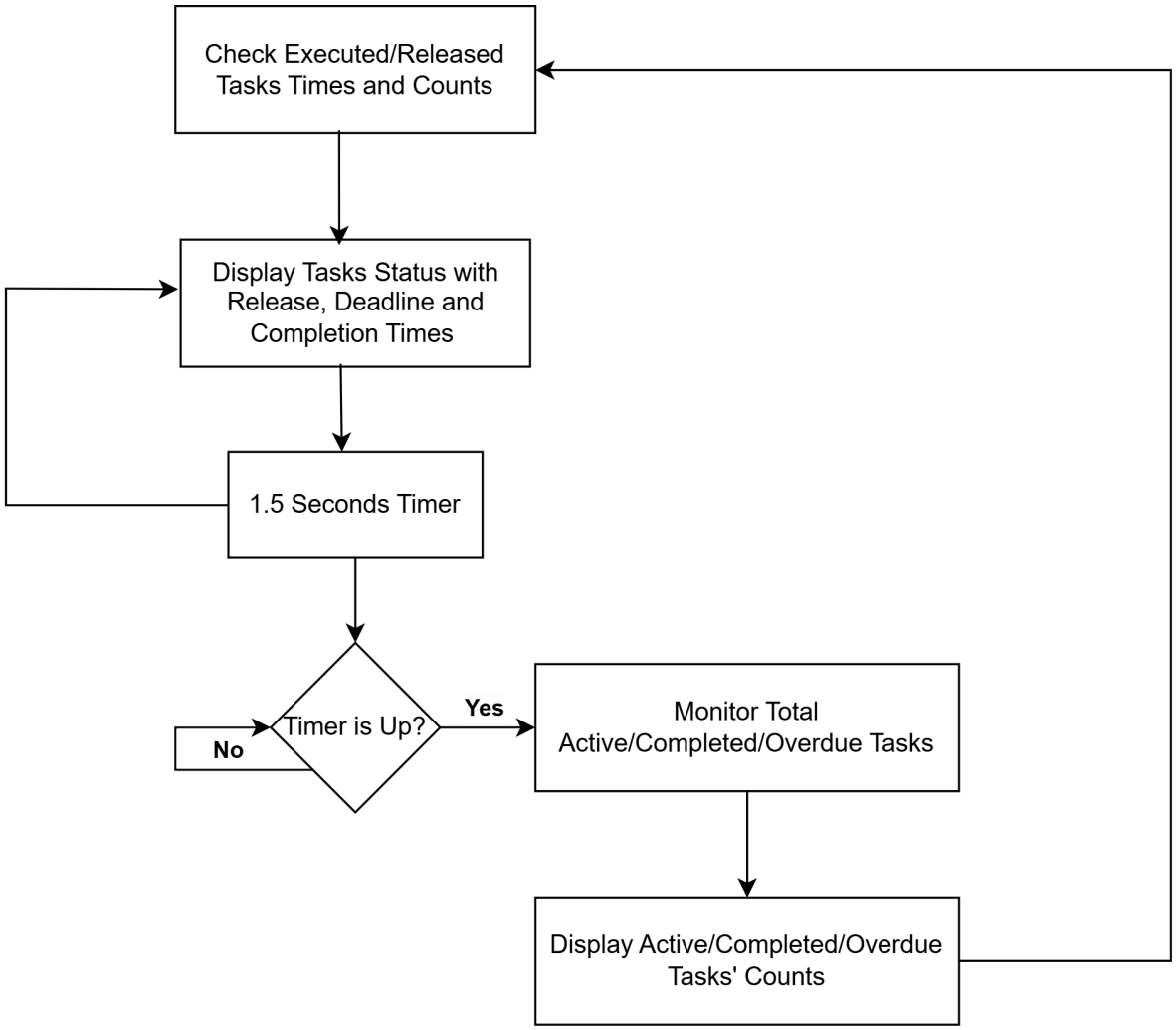

Diagram 2: Tasks Monitoring Flow Chart

After the execution and completion times are checked, the monitor task runs and starts a 1.5-second timer to print the total number of active, completed, and overdue tasks. The active task count is updated according to which task or tasks are currently running but have not yet finished execution, while the completed and overdue counts continue to increase as more tasks are identified as completed or overdue based on their completion time and absolute deadline. Once the task counts are displayed, the 1.5-second timer begins again, and the tasks in the active, completed, and overdue lists are displayed once more along with their status, release times, completion times, and absolute deadlines. This output is interrupted again when the 1.5-second timer expires, at which point the task counts are shown again.

2.2.3. Deadline Driven Task Sorting Algorithm

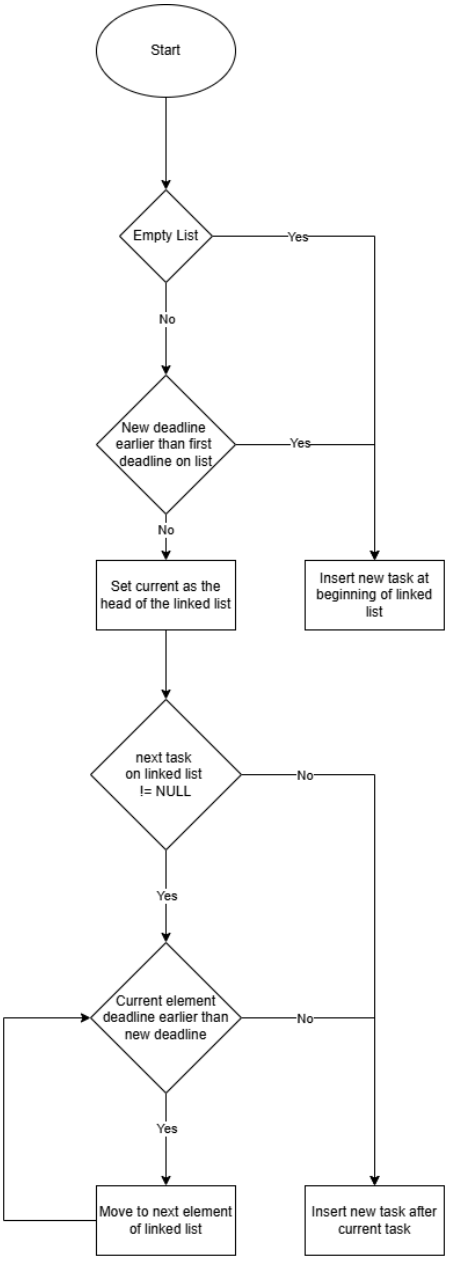

Diagram 3: Deadline driven task sorting algorithm.

Diagram 3 illustrates the deadline driven sorting algorithm for when a new task is inserted. For new tasks, if it is the first task or has the earliest deadline it is directly inserted at the beginning of the linked list. Otherwise, the linked list is traversed to find the first element that has a later deadline than the new task being added.

2.3. Testing Methodology

Table 2 shows the input data for each test bench used to verify the earliest deadline first scheduling of the DDS. The test benches start with all three tasks released at 0ms and the next occurrence of the task running periodically. Test bench 1 under utilizes the run time, test bench 2 over utilises the run time, and test bench 3 has 100% utilization. These factors will result in overdue tasks for test bench 2 and test bench 3. Figure 2 shows a sample output from test bench 1. The complete output is in Table 3.

	Test Bench #1		*Test Bench #2*		*Test Bench #3*	
Task	**Execution Time (ms)**	**Period (ms)**	**Execution Time (ms)**	**Period (ms)**	**Execution Time (ms)**	**Period (ms)**
t_1	95	500	95	250	100	500
t_2	150	500	150	500	200	500
t_3	250	750	250	750	200	500

Table 2: Test bench input data for verifying operation of the DDS [1].

```
Tasks Monitor:

T:1, Stat:Active, RT:0, ST:0, DT:500, CT:0

T:2, Stat:Active, RT:0, ST:0, DT:500, CT:0

T:1, Stat:Completed, RT:0, ST:0, DT:500, CT:95

A#: 2
C#: 1
O#: 0

T:3, Stat:Active, RT:0, ST:0, DT:750, CT:0

T:2, Stat:Completed, RT:0, ST:95, DT:500, CT:245

A#: 1
C#: 2
O#: 0

T:3, Stat:Completed, RT:0, ST:245, DT:750, CT:495

T:1, Stat:Active, RT:500, ST:0, DT:1000, CT:0

T:2, Stat:Active, RT:501, ST:0, DT:1001, CT:0

T:1, Stat:Completed, RT:500, ST:500, DT:1000, CT:595
```

Figure 2: Sample DDS output monitoring task lists and execution for the first 500ms of test bench 1.

2.3.1. Test Bench 1 (Under Utilization)

Test bench 1's theoretical schedule closely aligns with the expected timeline, but has some slight deviations by one ms for the completion and releases of a few tasks (Table 3). The completion of Task 2 in Event 10 and Event 16 deviates by one ms compared to the expected result. This coincides with the deviation of the Task 2 release times in Event 8 and Event 14; resulting in no overdue tasks by the scheduler. This deviation is expected in practice as the DDS tasks take computation time to run.

Event #	Event	Measured Time (ms)	Expected Time (ms)
1	Task 1 released	0	0
2	Task 2 released	0	0
3	Task 3 released	0	0
4	Task 1 complete	95	95
5	Task 2 complete	245	245
6	Task 3 complete	495	495
7	Task 1 released	500	500
8	**Task 2 released**	**501**	**500**
9	Task 1 complete	595	595
10	**Task 2 complete**	**746**	**745**
11	Task 3 released	750	750
12	Task 3 complete	1000	1000
13	Task 1 released	1000	1000
14	**Task 2 released**	**1001**	**1000**
15	Task 1 complete	1095	1095
16	**Task 2 complete**	**1246**	**1245**
17	Task 3 released	1500	1500
18	Task 1 released	1501	1500
19	Task 2 released	1502	1500

Table 3: Test bench 1 full hyperperiod (1500ms) run release and complete times.

2.3.2. Test Bench 2 (Over Utilization)

Test bench 2's theoretical schedule almost matches the measured, with only the number of active DD-Tasks deviating (Table 4). These three active tasks are due to the next hyper period of the scheduler starting. Therefore, in implementation it is expected that the scheduler instantly starts with these tasks as active and the monitor detects the changes at the 1500 ms second mark and after, which is outside the required demo period.

	Measured	**Expected**
Number of active DD-Tasks	3	3
Number of completed DD-Tasks	10	10
Number of overdue DD-Tasks	1	1

Table 4: Test bench 2 ending hyperperiod (1500ms) active, completed, and overdue tasks.

2.3.3. Test Bench 3 (100% Utilization)

Test bench 3's theoretical schedule has 100% utilization. Therefore, any delay added by the DDS causes overdue DD-Tasks in the schedule (Table 5). This deviates the monitor results from the theoretical zero overdue tasks. This is expected as the DDS adds overhead to the system.

	Measured	**Expected**
Number of active DD-Tasks	0	0
Number of completed DD-Tasks	8	9
Number of overdue DD-Tasks	1	0

Table 5: Test bench 3 ending hyperperiod (1500ms) active, completed, and overdue tasks.

3. Discussion

In this project, we explored the fundamental concepts involved in designing a real-time task scheduler. Key aspects included prioritizing tasks based on deadlines (with priority inversely proportional to deadline), as well as handling task preemption and suspension. FreeRTOS provides baseline task scheduling through a standard Round-Robin approach, where tasks are executed in a First-In-First-Out (FIFO) order within a queue. When a task's time slice expires or it is preempted, it is moved to the end of the queue for future execution.

To implement the Earliest Deadline First (EDF) algorithm on top of the Round-Robin scheduler, tasks are represented as messages and inserted into the main task queue according to their deadlines. In the worst-case scenario, this insertion process has a time complexity of O(n), such as when a new task has a deadline greater than all existing tasks. Once a task completes execution, it is removed from the active task queue, and the adjacent elements are reconnected to maintain list integrity.

A linked list structure was chosen to avoid the need for shifting elements during insertion and deletion operations. This design decision improves efficiency while minimizing memory usage. Additionally, inserting new tasks into the active task list according to their deadlines eliminates the need to re-sort the list whenever a new task is created. This improves performance, as it avoids the higher time complexity of sorting algorithms (O(n log n)) in favor of a simpler O(n) insertion operation.

The main challenge through the project is the manual managing of memory effectively to avoid overflow errors, system crashes, and incorrect data output. The tasks that required significant heap allocation were unit-tested extensively to ensure expected data within separate timeframes is always gathered without being corrupted by subsequent task calls.

Another challenge involved ensuring that only the active, completed, and overdue linked lists were passed correctly between tasks and helper functions without becoming corrupted. This required minimizing the risk of deleting incorrect tasks or overwriting the head pointers of the lists. Additionally, extensive integration testing was conducted between the task insertion and removal functions to verify that no errors occurred when creating new tasks or deleting completed ones.

When testing the scheduler with the three provided test batches, the task release, start, and completion times were within 1–2 ms of the expected values. These minor discrepancies are mainly due to scheduling overhead, particularly the context switching between the task generator, processor, and task monitor during task release periods. Additionally, since timers are not interrupted by task scheduling, this also contributes to the observed delay. Furthermore, the use of printf() to display the current task status introduces significant overhead. This is especially noticeable when critical sections are used to prevent interruptions during output. If a printf() call is still executing and another call is triggered by a newly scheduled task, the latter is deferred within FreeRTOS, leading to an additional delay of approximately 1–2 ms.

The results of all test batches were highly accurate and very close to the expected outcomes, tests' results were verified by sketching the tasks scheduling process on paper to ensure correctness.

As mentioned above, the limitations of the design were memory allocation and scheduling overhead. Many design decisions were made to reduce memory usage and scheduling overhead, but occasional simplifications were made due to the complexity of optimizations. Potential improvements include consolidating the "printf" statements executed before and after a task completes into a single call, which can significantly reduce processing overhead and lower the likelihood of missed deadlines by improving task execution efficiency. Additionally, removing one of the "printf" statements decreases the load on critical section handling, resulting in fewer operations for the scheduler to manage in each time slice. This is particularly beneficial since each "printf" requires a critical section to prevent interruptions and fragmented output during execution. The scheduler performance can also be improved by minimising the calls to the monitor task call through increasing the timer associated with displaying the total tasks counts within the active/completed/overdue list which further reduces the overhead associated with tasks' real-time tracking.

4. Summary

Throughout the project we have tackled how tasks scheduling can be significantly altered by changing the priorities of the tasks generator, processor, monitor, and currently executing tasks where slightest changes alter task preemption process and delay execution more than necessary. We have delved into the how efficient FreeRTOS can be when managing queues and critical sections while introducing minimal overhead to the overall performance reduction; furthermore, we have learned how task suspension can be a highly effective tool in managing the execution of max priority tasks and minimise preemption to avoid unnecessary delays.

The project also consisted of learning new methods in managing memory efficiently by using the heap dynamic memory allocation/freeing within tasks to avoid allocating large memory statically at once which can increase the risk of system crashing and degraded performance.

We have tackled how critical sections can be handled using fast binary/counting semaphores and a slow abstract critical section handling functionalities within FreeRTOS for different scenarios to ensure system reliability benefits outweigh the performance degradation when necessary and vice versa.

After testing the system, the measured release, start, and completion times of tasks in test batches 1 and 2 closely match the expected results, demonstrating high accuracy. In test batch 3, however, some tasks were marked as overdue due to a 1–2 millisecond delay caused by context switching between the task generator, processor, and monitor, as well as potential preemption of the currently executing task. This added processing overhead results in tasks completing slightly past their deadlines.

5. Appendix

The appendix includes the full code listing for the deadline driven scheduler (Code Listing A-1).

```
/* Authors: Bashar Jirjees - V00947950 &
   Nicholas McLennan -  V00989854*/

/* Importing the Needed Modules */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "stm32f4xx.h"
#include "stm32f4_discovery.h"
#include "stm32f4xx_adc.h"
#include "stm32f4xx_gpio.h"

#include "../FreeRTOS_Source/include/FreeRTOS.h"
#include "../FreeRTOS_Source/include/queue.h"
#include "../FreeRTOS_Source/include/semphr.h"
#include "../FreeRTOS_Source/include/task.h"
#include "../FreeRTOS_Source/include/timers.h"

/* Tasks Test Batch Number */
int chosen_test_batch = 3;

/* Defining Priorities and Delay Values*/
#define MAX_PRIORITY        4
#define HIGH_PRIORITY       3
#define LOW_PRIORITY        2
#define MIN_PRIORITY        1
#define QUEUE_DELAY 0XFFFFFFFF
#define SEMAPHORE_DELAY 0XFFFFFFFF

/* Defining the Fixed Task Types */
typedef enum {PERIODIC, APERIODIC} task_type;

/* Defining the Fixed Message Types */
typedef enum {RELEASE, COMPLETE, ACTIVE_TASKS, COMPLETED_TASKS,
OVERDUE_TASKS} message_type;

/* Defining the Task Contents */
struct dd_task{
    TaskHandle_t t_handle;
    task_type type;
    uint32_t task_id;
    uint32_t release_time;
    uint32_t absolute_deadline;
    uint32_t completion_time;
};

/* Defining the Task List Contents */
struct dd_task_list{
    struct dd_task task;
```

```
    struct dd_task_list *next_task;
};


/* Defining the Message Contents */
typedef struct {
    message_type m_type;
    TaskHandle_t t_handle;
    task_type type;
    uint8_t task_id;
    uint32_t completion_time;
    uint32_t task_counter_id;
    uint32_t absolute_deadline;
    struct dd_task_list* task_list;
} task_message;

/* Defining the Task Type Lists */
struct dd_task_list* active_list_head = NULL;
struct dd_task_list* completed_list_head = NULL;
struct dd_task_list* overdue_list_head = NULL;

/* Defining the Queues that Stores All Tasks */
QueueHandle_t tasks_queue;
QueueHandle_t prev_exec;
QueueHandle_t lists_queue;

/* Defining the Generator and Monitor Tasks Variables */
TaskHandle_t generator_handle;
TaskHandle_t monitor_handle;

/* List Used to Index the Status of the Completed Task Based on
Completion Time and Deadline */
char *completed_overdue[2] = {"Completed", "Overdue"};

/*
    This Function Takes the Newly Generated Task and Adds it to the All
    Tasks Queue
    The New Task Queue Location Depends on Its Deadline, The Shorter
    the Deadline,
    the Closer the New Task to Position 0 in the Queue (Less Time to
    Pop from the Queue)

    Input: Active Tasks List and the Newly Added Task
    Return: None
*/

void insert_task(struct dd_task_list** head, struct dd_task_list*
                 new_task) {

    struct dd_task_list* current;
    if (*head == NULL || (*head)->task.absolute_deadline >
         new_task->task.absolute_deadline) {
        new_task->next_task = *head;
        *head = new_task;
    } else {
        current = *head;
        while (current->next_task != NULL &&
        current->next_task->task.absolute_deadline <=
```

```
        new_task->task.absolute_deadline) {
            current = current->next_task;
        }
        new_task->next_task = current->next_task;
        current->next_task = new_task;
    }
}

/*
    This Function Removes a Specific Completed Task Based on Its ID
    From the Active Tasks List
    The Tasks Doesn't Perform any Sorting

    Input: Active Tasks List and the Completed Task ID
    Return: None
*/

struct dd_task_list* remove_task(struct dd_task_list** head, uint32_t
id) {

    struct dd_task_list* temp = *head;
    struct dd_task_list* prev = NULL;

    if (temp != NULL && temp->task.task_id == id) {
        *head = temp->next_task;
        temp->next_task = NULL;
        return temp;
    }
    while (temp != NULL && temp->task.task_id != id) {
        prev = temp;
        temp = temp->next_task;
    }
    if (temp == NULL) return NULL;
    prev->next_task = temp->next_task;
    temp->next_task = NULL;

    return temp;
}

/*
    This Function Sets the Task with the Lowest Deadline as the Highest
    Priority and
    Other Tasks as Low Priority; Low Priority > Min Priority

    Input: None
    Return: None
*/

void update_tasks_priorities(void) {

    struct dd_task_list* current = active_list_head;
    if (current != NULL) {

        vTaskPrioritySet(current->task.t_handle, HIGH_PRIORITY);
        if(eTaskGetState(current->task.t_handle) == eSuspended ||
           eTaskGetState(current->task.t_handle) == eBlocked)
            vTaskResume(current->task.t_handle);
```

```
        current = current->next_task;
        while (current != NULL) {

            vTaskPrioritySet(current->task.t_handle, MIN_PRIORITY);
            vTaskSuspend(current->task.t_handle);

            current = current->next_task;
        }
    }
}

/*
    This Function Creates Structure Based on the Newly Generated Task
    Parameters and
    Adds it to the All Tasks Queue

    Input: Newly Generated Task, Task Type, Task ID, and Task Execution
    Deadline
    Return: None
*/

void create_dd_task(TaskHandle_t t_handle, task_type type, uint32_t
task_counter_id, uint32_t task_id, uint32_t absolute_deadline) {

    task_message msg;
    msg.m_type = RELEASE;
    msg.t_handle = t_handle;
    msg.type = type;
    msg.task_id = task_id;
    msg.task_counter_id = task_counter_id,
    msg.absolute_deadline = absolute_deadline;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(QUEUE_DELAY));
}

/*
    This Function Adds the Completed Tasks Again to The All Tasks Queue
    But with Complete State
    To be Removed

    Input: Task ID
    Return: None
*/

void delete_dd_task(uint32_t task_counter_id, uint8_t task_id, uint32_t
completion_time, uint32_t deadline) {

    task_message msg;

    msg.m_type = COMPLETE;
    msg.task_id = task_id;
    msg.completion_time = completion_time;
    msg.absolute_deadline = deadline;
    msg.task_counter_id = task_counter_id;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(QUEUE_DELAY));
}
```

```c
/*
    This Function Returns the List of the Currently Active Tasks

    Input: None
    Return: The Active Tasks List Head
*/

struct dd_task_list* get_active_dd_task_list(void) {

    task_message msg;
    struct dd_task_list* active_list_head;
    msg.m_type = ACTIVE_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &active_list_head, pdMS_TO_TICKS(100));

    return active_list_head;
}

/*
    This Function Returns the List of the Currently Completed Tasks

    Input: None
    Return: The Completed Tasks List Head
*/

struct dd_task_list* get_completed_dd_task_list(void) {

    task_message msg;
    struct dd_task_list* completed_list_head;
    msg.m_type = COMPLETED_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &completed_list_head,
    pdMS_TO_TICKS(100));

    return completed_list_head;
}

/*
    This Function Returns the List of the Currently Overdue Tasks

    Input: None
    Return: The Overdue Tasks List Head
*/

struct dd_task_list* get_overdue_dd_task_list(void) {

    task_message msg;
    struct dd_task_list* overdue_list_head;
    msg.m_type = OVERDUE_TASKS;

    xQueueSend(tasks_queue, &msg, pdMS_TO_TICKS(0));
    xQueueReceive(lists_queue, &overdue_list_head, pdMS_TO_TICKS(100));

    return overdue_list_head;
}
```

```
/*
    This Function Display the Tasks Status, Release Time, Deadline, and
    Completion Time
    The function Also Summarizes the Total Number of Active, Completed
    and Overdue Tasks

    Input: Active/Completed/Overdue Tasks List and Task Fixed Status
    with That List
    Return: None
*/

void get_current_tasks(struct dd_task_list* head, char* task_status) {

        struct dd_task_list* current = head;

        uint32_t count = 0;

        char *total_count_status[] = {"GET_ACTIVE_TASK_COUNT",
"GET_COMPLETED_TASK_COUNT", "GET_OVERDUE_TASK_COUNT"};

        while (current != NULL)
        {
            // Checking that the Semaphore is Available Before
            // Increment Over Counting
            ++count;
            current = current->next_task;

        }

       if (strcmp(task_status , total_count_status[0]) == 0)
printf("A#: %i\n", (int)count);
       else if (strcmp(task_status , total_count_status[1]) == 0)
printf("C#: %i\n", (int)count);
       else if (strcmp(task_status , total_count_status[2]) == 0)
printf("O#: %i\n", (int)count);
}

/*
    This Function Checks the Tasks Status Regularly to Adjust their
    Position
    Within the Tasks List and Alters the Tasks Current Status based on
    Completion Time

    Input: None
    Return: None
*/

void vDDSTask(void *pvParameters) {

    task_message msg;
    uint32_t current_time;

    while (1) {

        //  Getting the Next Released Task Message in The Queue
        if (xQueueReceive(tasks_queue, &msg,
            pdMS_TO_TICKS(QUEUE_DELAY)) == pdTRUE){
```

```
current_time = xTaskGetTickCount();

// If a New Task is Released then it's Added to the
// Active Tasks List
if(msg.m_type == RELEASE) {

    struct dd_task_list* new_task =
    pvPortMalloc(sizeof(struct dd_task_list));

    new_task->task.t_handle = msg.t_handle;
    new_task->task.type = msg.type;
    new_task->task.task_id = msg.task_counter_id;
    new_task->task.release_time = current_time;
    new_task->task.absolute_deadline = current_time +
    pdMS_TO_TICKS(msg.absolute_deadline);
    new_task->task.completion_time = 0;
    new_task->next_task = NULL;

    insert_task(&active_list_head, new_task);

    // Ensuring the Tasks Queue is Empty Before
    // Updating New Tasks Priorities
    if(xQueuePeek(tasks_queue, &msg, pdMS_TO_TICKS(0))
       == pdFALSE)
        update_tasks_priorities();
}

// If a Task is Completed then it's Added to the
// Completed/Overdue Tasks List
// The Completed Task is Removed from the Active Tasks
// List
// The Completed Task is Also Checked for Completing
// After Deadline
// Newly Released Tasks Can Start Executing
else if(msg.m_type == COMPLETE) {

    struct dd_task_list* completed_task =
    remove_task(&active_list_head,
    msg.task_counter_id);

    if (completed_task != NULL){

        completed_task->task.completion_time =
        msg.completion_time;

        if (completed_task->task.completion_time >
            msg.absolute_deadline) {
            completed_task->next_task =
            overdue_list_head;
            overdue_list_head = completed_task;

        }else {
            completed_task->next_task =
            completed_list_head;
            completed_list_head = completed_task;
        }
    }
    update_tasks_priorities();
```

```
            // Checking if the Counts for the Current Tasks in the
            // Active/Complete/Overdue
            // are Required to Be Printed to the Console for Constant
            // Monitoring
            }else if(msg.m_type == ACTIVE_TASKS) {
                xQueueSend(lists_queue, &active_list_head,
                pdMS_TO_TICKS(0));
            }else if (msg.m_type == COMPLETED_TASKS) {
                xQueueSend(lists_queue, &completed_list_head,
                pdMS_TO_TICKS(0));
            }else if (msg.m_type == OVERDUE_TASKS) {
                xQueueSend(lists_queue, &overdue_list_head,
                pdMS_TO_TICKS(0));
            }
        }
    }
}

/*
     Temporary Structure Used for Communicating the Test Batch Task
     Info with Other Tasks
*/

typedef struct{

    uint8_t taskid;
    uint32_t task_counter_id;
    uint32_t release_time;
    uint16_t delay;
    uint16_t deadline;
    TaskHandle_t t_handle;

} task_params;

/*
    Task Handles to Control the Task Generator, Task Checker, and
    Monitor Functionality and
    Execution
*/

TaskHandle_t generator_handle;
TaskHandle_t monitor_handle;
TaskHandle_t dd_task_handle;

/*
    This Function Represents the Body for The Executing Task
    The Functionality is Simply a Very Short Delay that Simulates
    Execution Time

    Input: Task Parameters in Structure Type "task_params"
    Return: None
*/

void Execute_Task(void *pvParameters) {
```

```
    task_params *task_params_temp = (task_params *)pvParameters;

    uint32_t task_counter_id = task_params_temp->task_counter_id;
    uint32_t start_time = xTaskGetTickCount();
    uint32_t release_time = task_params_temp->release_time;
    uint32_t task_deadline = release_time + task_params_temp->deadline;
    uint16_t ticks_counter = 0;
    uint8_t task_id = task_params_temp->taskid;

    taskENTER_CRITICAL();

    printf("\nT:%i, Stat:%s, RT:%i, ST:%i, DT:%i, CT:%i\n",
                                    (int)task_id,
                                    "Active",
                                    (int)release_time,
                                    (int)0,
                                    (int)task_deadline,
                                    0);
    taskEXIT_CRITICAL();

    uint32_t endtime = xTaskGetTickCount();
    uint32_t prev_tick = xTaskGetTickCount();

    // Ensuring that the Number of Ticks Between Start Time and Finish
    // Time is Always
    // Bigger or Equal to the Task Delay
    while(1 == 1) {

        uint32_t tick = xTaskGetTickCount();

        if (tick > prev_tick) ++ticks_counter;

        prev_tick = tick;

        // Ensuring the Total Delay is Accounting for the Delay of
        // Printing the Task Stats
        // in the Prior "Printf" Statement
        if (ticks_counter >=  (task_params_temp->delay - (endtime -
            start_time))) break;
    };

    // Getting the Real Task Completion Time
    uint32_t delay = xTaskGetTickCount();

    //Flag The Task For Deletion After Completion
    delete_dd_task(task_counter_id, task_id, delay, task_deadline);

    taskENTER_CRITICAL();

    printf("\nT:%i, Stat:%s, RT:%i, ST:%i, DT:%i, CT:%i\n",
                                (int)task_id,
                                completed_overdue[task_deadline >=
                                delay ? 0 : 1],
                                (int)release_time,
                                (int)start_time,
                                (int)task_deadline,
                                (int)delay);
```

```
    taskEXIT_CRITICAL();


    // Delete the Task After Completion
    vTaskDelete(NULL);

    vPortFree(task_params_temp);
}

/*
    Temporary Queue Used to Determine which Task's Timer has Currently
    Finished,
    which ensures the Correct Test Task Parameters are Used for
    Periods, Execution Time, and
    Monitoring Tasks Executions
*/

QueueHandle_t timers_task_released;

/*
    Mutex Needed to Generate Tasks Based on Timers and Not Continuously
*/

SemaphoreHandle_t Mutex_Generator;

/*
    These Functions Times the Generation of New Task to ensure Periodic
    Tasks are Released
    After Specific Time Intervals

    Input: Timer Structure
    Return: None
*/

void vTask_1_Timer_Callback(TimerHandle_t xTimer) {

    uint8_t task = 1;
    xQueueSend(timers_task_released, &task,
    pdMS_TO_TICKS(QUEUE_DELAY));

    xSemaphoreGive(Mutex_Generator);

}

void vTask_2_Timer_Callback(TimerHandle_t xTimer) {

    uint8_t task = 2;
    xQueueSend(timers_task_released, &task,
    pdMS_TO_TICKS(QUEUE_DELAY));

    xSemaphoreGive(Mutex_Generator);

}

void vTask_3_Timer_Callback(TimerHandle_t xTimer) {

    uint8_t task = 3;
    xQueueSend(timers_task_released, &task,
```

```
    pdMS_TO_TICKS(QUEUE_DELAY));

    xSemaphoreGive(Mutex_Generator);
}

/*
    Helper Task the vDDTaskGenerator to Avoid Repetitive Ode when
    Generating Tasks at Time 0
    and Periodically After

    Input: Task ID (Between 1 and 3), Task Unique ID, Task Execution
    and Period Times Defined in the Test Batch
    Return: None
*/

void GenerateTasks(uint8_t task_id, uint32_t counter, uint16_t
chosen_test_executions[], uint16_t chosen_test_periods[], task_type
msg_type
){

    task_params *task_params_temp = pvPortMalloc(sizeof(task_params));

    uint16_t deadline = msg_type == PERIODIC ?
                        chosen_test_periods[task_id - 1] : 0xFFFF;

    task_params_temp->taskid = task_id;
    task_params_temp->task_counter_id = counter;
    task_params_temp->delay = chosen_test_executions[task_id - 1];
    task_params_temp->deadline = chosen_test_periods[task_id - 1];
    task_params_temp->release_time = xTaskGetTickCount();

    TaskHandle_t TaskHandle = NULL;

    xTaskCreate(Execute_Task, "task", 128, (void *) task_params_temp,
                0, &TaskHandle);
    vTaskSuspend(TaskHandle);

    task_params_temp->t_handle = TaskHandle;

    create_dd_task(TaskHandle, msg_type,
    task_params_temp->task_counter_id, task_params_temp->taskid,
    task_params_temp->deadline);
}

/*
    This Function Generates New Task within Specific Time Intervals and
    Adds them to the Active Tasks List

    Input: None
    Return: None
*/

void vDDTaskGenerator(void *pvParameters) {

    Mutex_Generator = xSemaphoreCreateCounting(3,0);

    // Test Batches Parameters
    uint16_t test_1_periods[3] = {500, 500, 750};
```

```
    uint16_t test_2_periods[3] = {250, 500, 750};
    uint16_t test_3_periods[3] = {500, 500, 500};

    uint16_t test_1_executions[3] = {95, 150, 250};
    uint16_t test_2_executions[3] = {95, 150, 250};
    uint16_t test_3_executions[3] = {100, 200, 200};

    uint16_t chosen_test_periods[3];
    uint16_t chosen_test_executions[3];

    if (chosen_test_batch == 1) {
        memcpy(chosen_test_periods, test_1_periods,
        sizeof(test_1_periods));
        memcpy(chosen_test_executions, test_1_executions,
        sizeof(test_1_executions));
    }
    else if(chosen_test_batch == 2){
        memcpy(chosen_test_periods, test_2_periods,
        sizeof(test_2_periods));
        memcpy(chosen_test_executions, test_2_executions,
        sizeof(test_2_executions));
    }
    else if(chosen_test_batch == 3){
        memcpy(chosen_test_periods, test_3_periods,
        sizeof(test_3_periods));
        memcpy(chosen_test_executions, test_3_executions,
        sizeof(test_3_executions));
    }else{
        perror("Input Correct Test Batch Number Between 1 and 3\n");
        exit(EXIT_FAILURE);
    }

    timers_task_released = xQueueCreate(1, sizeof(uint8_t));

    // Task Counter ID
    uint32_t counter = 2;

    // Tasks Timers Used to Periodically Generate Tasks and Add them to
    // the Tasks Queue
    TimerHandle_t xTimer_1;
    TimerHandle_t xTimer_2;
    TimerHandle_t xTimer_3;

    xTimer_1 = xTimerCreate("Task_1_Timer",
    pdMS_TO_TICKS(chosen_test_periods[0]), pdTRUE, NULL,
    vTask_1_Timer_Callback);
    xTimer_2 = xTimerCreate("Task_2_Timer",
    pdMS_TO_TICKS(chosen_test_periods[1]), pdTRUE, NULL,
    vTask_2_Timer_Callback);
    xTimer_3 = xTimerCreate("Task_3_Timer",
    pdMS_TO_TICKS(chosen_test_periods[2]), pdTRUE, NULL,
    vTask_3_Timer_Callback);

    GenerateTasks(1, 0, chosen_test_executions,
                  chosen_test_periods,PERIODIC);
    GenerateTasks(2, 1, chosen_test_executions, chosen_test_periods,
                  PERIODIC);
    GenerateTasks(3, 2, chosen_test_executions, chosen_test_periods,
```

```
                    PERIODIC);

    xTimerStart(xTimer_1, 0);
    xTimerStart(xTimer_2, 0);
    xTimerStart(xTimer_3, 0);

    uint8_t task_id;

    while (1) {

        if (xSemaphoreTake(Mutex_Generator,
            pdMS_TO_TICKS(SEMAPHORE_DELAY)) == pdTRUE){

            // Ensuring the Task is Never Preempted when Adding  newly
            // Generated Task to the Tasks Queue,
            // Avoiding Task Ordering Problems
            taskENTER_CRITICAL();

            ++counter;

            xQueueReceive(timers_task_released, &task_id,
            pdMS_TO_TICKS(QUEUE_DELAY));

            GenerateTasks(task_id, counter, chosen_test_executions,
            Chosen_test_periods, PERIODIC);

            // Resuming Preemption
            taskEXIT_CRITICAL();
        }
    }
}

/*
    Mutexes that Ensure Printing Messages Don't Happen Until Previous
    Ones Finish Executing,
    Minimizing Overlapping Between Statements and Printing Total Tasks
    Counts 200 Milliseconds
*/

SemaphoreHandle_t Mutex_Monitor_Timer = NULL;

/*
    This Functions Times the Print Statement for the Total Tasks Counts
    in Active, Overdue and Completed States

    Input: Timer Structure
    Return: None
*/

void vTotal_Counts_Timer_Callback(TimerHandle_t xTimer) {

    xSemaphoreGive(Mutex_Monitor_Timer);
}



/*
    This Function Prints Out the Current Status of Tasks in Real-time
```

```
    Input: None
    Output: None
*/

void vMonitorTask(void *pvParameters) {

    Mutex_Monitor_Timer = xSemaphoreCreateBinary();


    // Timer is Used to Report the Total Number of Tasks Every 200
    // Milliseconds
    TimerHandle_t xTimer_Total_Counts =
    xTimerCreate("Total_Counts_Timer", pdMS_TO_TICKS(200),
                 pdTRUE, NULL,
    vTotal_Counts_Timer_Callback);

    xTimerStart(xTimer_Total_Counts, 0);


    while (1) {

        //Checking the Timer has Been Triggered
        if (xSemaphoreTake(Mutex_Monitor_Timer,
            pdMS_TO_TICKS(SEMAPHORE_DELAY)) == pdTRUE){

            printf("\n");
            get_current_tasks(get_active_dd_task_list(),
            "GET_ACTIVE_TASK_COUNT");
            get_current_tasks(get_completed_dd_task_list(),
            "GET_COMPLETED_TASK_COUNT");
            get_current_tasks(get_overdue_dd_task_list(),
            "GET_OVERDUE_TASK_COUNT" );

        }
    }
}

/*
    This is the Main Function, It Specified the Tasks Needed for
    Scheduling and Initiates
    the FreeRtos Scheduler

    Input: None
    Output: None
*/

int main(void) {

    tasks_queue = xQueueCreate(10, sizeof(task_message));
    lists_queue = xQueueCreate(10, sizeof(struct dd_task_list));

    if (tasks_queue != NULL && lists_queue != NULL)
    {

        printf("\nTasks Monitor:\n");

        //Defining and Initiating the Tasks for FreeRTOS
```

```
        xTaskCreate(vDDTaskGenerator, "Task_Generator", 256, NULL,
        MAX_PRIORITY, &generator_handle);
        xTaskCreate(vDDSTask, "Task_Checker", 128, NULL, MAX_PRIORITY,
        &dd_task_handle);
        xTaskCreate(vMonitorTask, "Task_Monitor", 128, NULL,
        MAX_PRIORITY, &monitor_handle);

        vTaskStartScheduler();
    }

    while(1);

    return 0;
}

/*-----------------------------------------------------------*/

void vApplicationMallocFailedHook( void )
{
    /* The malloc failed hook is enabled by setting
    configUSE_MALLOC_FAILED_HOOK to 1 in FreeRTOSConfig.h.

    Called if a call to pvPortMalloc() fails because there is
    insufficient
    free memory available in the FreeRTOS heap.  pvPortMalloc() is
    called
    internally by FreeRTOS API functions that create tasks, queues,
    software
    timers, and semaphores.  The size of the FreeRTOS heap is set by
    the configTOTAL_HEAP_SIZE configuration constant in
    FreeRTOSConfig.h. */
    for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationStackOverflowHook( xTaskHandle pxTask, signed char
*pcTaskName )
{
    ( void ) pcTaskName;
    ( void ) pxTask;

    /* Run time stack overflow checking is performed if
    configconfigCHECK_FOR_STACK_OVERFLOW is defined to 1 or 2. This
    Hook function is called if a stack overflow is detected.
    pxCurrentTCB can be
    inspected in the debugger if the task name passed into this
    function is corrupt. */
    for( ;; );
}
/*-----------------------------------------------------------*/

void vApplicationIdleHook( void )
{
volatile size_t xFreeStackSpace;


    /* The idle task hook is enabled by setting configUSE_IDLE_HOOK to
    1 in FreeRTOSConfig.h.
```

```
    This function is called on each cycle of the idle task.  In this
    case it
    does nothing useful, other than report the amount of FreeRTOS heap
    that remains unallocated. */
    xFreeStackSpace = xPortGetFreeHeapSize();

    if( xFreeStackSpace > 100 )
    {
        /* By now, the kernel has allocated everything it is going to,
        so if there is a lot of heap remaining unallocated then
        the value of configTOTAL_HEAP_SIZE in FreeRTOSConfig.h can be
        reduced accordingly. */
    }
}
/*-----------------------------------------------------------*/

static void prvSetupHardware( void )
{
    /* Ensure all priority bits are assigned as preemption priority
       bits.
    http://www.freertos.org/RTOS-Cortex-M3-M4.html */
    NVIC_SetPriorityGrouping( 0 );

    /* TODO: Setup the clocks, etc. here, if they were not configured
       before
    main() was called. */
}
```

6. References

1. University of Victoria, ECE455 Lab Manual, Victoria, BC: University of Victoria, 2019. https://bright.uvic.ca/d2l/le/content/429212/viewContent/3560930/View
2. FreeRTOS.org, “FreeRTOS documentation - freertosTM,” FreeRTOS, https://www.freertos.org/Documentation/00-Overview